

HPML

High-Performance Machine Learning

Final Exam — Complete Study Notes

Course	COMS E6998 — High-Performance Machine Learning
University	Columbia University — Spring 2026
Instructor	Dr. Kaoutar El Maghraoui
Exam Date	Monday, May 11, 2026 · 7:00 PM – 9:00 PM
Location	Schermerhorn Hall, Room 614
Format	Closed book · One cheat sheet (8.5×11, both sides, ≥8pt font)
Weight	20% of final grade · ~25–35 questions
Generated	May 09, 2026

Exam Coverage by Tier

Lecture	Topic	Tier
L3	PyTorch Performance and Profiling	Tier 1 (Heavy — ~70%)
L4	CUDA Basics	Tier 1 (Heavy — ~70%)
L5	Advanced CUDA	Tier 1 (Heavy — ~70%)
L6	Distributed Deep Learning	Tier 1 (Heavy — ~70%)
L9	Efficient Transformers & FlashAttention	Tier 1 (Heavy — ~70%)
L1	Intro HPC & AI (Amdahl, scaling)	Tier 2 (~25%)
L2	Performance Methodology & Optimizers	Tier 2 (~25%)
L7	Quantization (PTQ vs QAT, INT8/FP8)	Tier 2 (~25%)
L8	Pruning & Sparsity	Tier 2 (~25%)

L10	Knowledge Distillation	Tier 2 (~25%)
-----	------------------------	---------------

CRITICAL NUMBERS TO MEMORIZE

★ EXAM: These numbers appear directly in exam questions. Know them cold.

GPU & CUDA Constants

Item	Value	Why It Matters
Warp size	32 threads	Basic scheduling unit; warp divergence is costly
Max threads per block	1024	Cannot exceed when setting blockDim
Shared memory banks	32 banks, 4-byte granule	Bank conflicts degrade performance
Common block sizes	128, 256, 512	Multiples of 32; maximize occupancy
HBM latency (A100)	~400–600 cycles	Motivates caching in shared memory
SRAM (shared mem) BW	~19 TB/s	~13× faster than HBM
HBM bandwidth (A100)	~1.5–2 TB/s (HBM2e)	Bottleneck for memory-bound kernels
HBM bandwidth (H100)	~3.35 TB/s (HBM3)	Still the bottleneck
SRAM (on-chip) capacity	~20 MB per GPU	FlashAttention tiles into this
HBM capacity (A100/H100)	~40–80 GB	Limits model size per GPU
A100 CUDA cores	6,912	Ampere architecture
H100 CUDA cores	18,432	Hopper architecture — 2.7× more
A100 Tensor cores	432 (3rd gen)	FP16/TF32/INT8
H100 Tensor cores	640 (4th gen)	Adds FP8
A100 L2 cache	40 MB	Chip-wide shared cache
H100 L2 cache	50 MB	Larger for bigger models

Memory Precision Reductions

Conversion	Memory Reduction	Use Case
FP32 → FP16	2×	Training with mixed precision
FP32 → BF16	2×	LLM pre-training (same range as FP32)
FP32 → INT8	4×	Inference quantization
FP32 → INT4	8×	Aggressive inference quantization
FP32 → FP8 (E4M3/E5M2)	4×	H100 training (forward/backward)
FP32 → FP4	8×	Blackwell (B200) inference

FlashAttention Key Numbers

Item	Value
Standard attention memory	$O(N^2)$ — stores full $N \times N$ matrix in HBM
FlashAttention HBM I/O	$O(N^2 \cdot d^2 / M)$ — M = SRAM size; d = head dim
FlashAttention FLOPs	$O(N^2 \cdot d)$ — same as standard attention
Wall-clock speedup	2–4× (up to 6× with masking+dropout)
Memory reduction	10–20×
Block size B_c	$\lceil M / 4d \rceil$
Block size B_r	$\min(\lceil M / 4d \rceil, d)$
Standard attention runtime (GPT-2 medium)	41.7 ms
FlashAttention runtime (GPT-2 medium)	7.3 ms (5.7× faster)
HBM R/W standard	40.3 GB
HBM R/W FlashAttention	4.4 GB (~9× less)

Distributed Training Key Numbers

Item	Value
Ring AllReduce comm cost per GPU	$2N(P-1)/P \approx 2N$ for large P
Parameter Server comm cost	$2N(P-1)$ — scales poorly with P
LLaMA 3 405B memory (fp16)	810 GB — needs 10+ H100s just for weights
GPT-3 175B memory w/ Adam	~2.8 TB (16 bytes/param)
ZeRO-1 max model ($N=64$ GPUs)	~19B params
ZeRO-2 max model ($N=64$ GPUs)	~36B params
ZeRO-3 max model ($N=64$ GPUs)	~320B params
DDP overhead vs ideal	~5–15% from AllReduce
TSM video model speedup	50 hrs → 14 min with 1,536 GPUs (211×)

L1 & L2 — HPC Intro, Performance Methodology & Optimizers

HPML Definition & Course Structure

HPML = The science of making AI **Fast, Efficient, and Scalable**

Module	Topics
I. Foundations	HPC/AI Intro, PyTorch Fundamentals, Profiling
II. GPU Computing	CUDA Basics, Advanced CUDA, Distributed DL
III. Compression	Pruning, Quantization, Knowledge Distillation
IV. Efficient LLMs	FlashAttention, vLLM, Speculative Decoding, LoRA
V. NAS & Projects	Neural Architecture Search, Final Presentations

Roofline Model

A throughput-based performance model with three key metrics:

- **Peak FLOPS** = #cores × frequency × FLOP/cycle
- **Memory Bandwidth** = data/time [GB/s]
- **Arithmetic Intensity (AI)** = #arithmetic_ops / DRAM_bytes [FLOP/byte]

$$\text{Actual FLOPS} = \min(\text{Peak FLOPS}, \text{AI} \times \text{Memory BW})$$

- Low AI → memory-bound → optimize data reuse (cache blocking)
- High AI → compute-bound → use SIMD/vectorization
- **DAXPY example:** 2 FLOP / 24 bytes = 0.083 FLOP/byte → memory-bound
- **CrossPoint:** Peak FLOPS / Memory BW = threshold (e.g., 3.17 FLOP/byte for Xeon E5630)

Amdahl's Law

$$S(p, s) = 1 / [(1 - p) + p/s]$$

- S = speedup of entire app; p = fraction of time in improved section; s = speedup of that section
- Overall speedup is limited by the serial fraction

★ **EXAM: Always optimize the section with the highest p (critical path) first**

Performance Optimization Cycle

Step 1 MEASURE → Execute workload, profile, trace, time

Step 2 ANALYZE → Identify critical path + bottleneck via Roofline

Step 3 OPTIMIZE → Apply correct technique based on bottleneck type

Key Means for Performance Benchmarking

Mean Type	Formula	Best For
Arithmetic	$(1/n)\sum x_i$	Execution time, latency, CPU performance
Harmonic	$n / \sum (1/x_i)$	Throughput, rates, network BW, F1 score
Geometric	$(\prod x_i)^{(1/n)}$	Speedup ratios, benchmarking comparisons

Scaling Definitions

- **Strong Scaling:** Fixed total problem size, add more GPUs → time decreases. Efficiency = $t_{\text{single}} / (N \times t_N)$
- **Weak Scaling:** Problem size grows proportionally with GPUs → runtime stays constant

★ EXAM: DDP with fixed global batch + 1→8 GPUs = **STRONG** scaling (same total work, more GPUs)

Memory Hierarchy Latencies

Level	Latency	Notes
Registers	1 cycle	Fastest; private to thread
L1 Cache	~3 cycles	On-chip, per SM
L2 Cache	~10 cycles	On-chip, shared
L3 Cache	~75 cycles	Last level CPU cache
DRAM	~100 cycles	Main memory
InfiniBand	2000+ cycles	Network
GPU mem (PCIe)	~5,000 cycles	Host→Device transfer
Ethernet	20,000+ cycles	Slow; not for ML clusters

Optimizer Formulas (Cheat Sheet)

Vanilla SGD

$$w_{t+1} = w_t - \alpha (\partial L / \partial w_t)$$

Momentum

$$v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot (\partial L / \partial w_t) ; w_{t+1} = w_t - \alpha \cdot v_t$$

Adam (most important)

$$\begin{aligned} v_t &= \beta_1 \cdot v_{t-1} + (1-\beta_1) \cdot g_t \quad [1\text{st moment} / \text{momentum}] \\ s_t &= \beta_2 \cdot s_{t-1} + (1-\beta_2) \cdot g_t^2 \quad [2\text{nd moment} / \text{RMSProp}] \\ \hat{v}_t &= v_t / (1-\beta_1^t) ; \hat{s}_t = s_t / (1-\beta_2^t) \quad [\text{bias-corrected}] \\ w_{t+1} &= w_t - \alpha \cdot \hat{v}_t / (\sqrt{\hat{s}_t} + \epsilon) \end{aligned}$$

AdamW (de facto LLM standard)

Same as Adam but weight decay applied directly: $w_{t+1} = w_{t+1, \text{Adam}} - \lambda \cdot w_t$

Decouples weight decay from adaptive gradient update. Used in LLaMA, GPT-3, etc.

Optimizer	LR	β_1	β_2	Used By
AdamW	3e-4 (cosine)	0.9	0.95	LLaMA (Meta)
Adam (custom)	6e-4	0.9	0.98	GPT-3 (OpenAI)
Lion	2e-3	0.95	0.98	DeepSeek-67B
SGD+momentum	varies	0.9	—	Vision models

Training vs Inference

Aspect	Training	Inference
Goal	Maximize throughput	Minimize latency
Memory	Weights + Gradients + Optimizer states	Weights + KV Cache
Compute	Forward + Backward (backward $\approx 2\times$ forward)	Forward pass only
Key metric	Time-to-accuracy	Tokens/sec (TPS)
Phases	Full training loop	Prefill (KV cache) + Decode (autoregressive)
Bound	Compute-bound	Memory-bandwidth-bound (decode)

★ EXAM: Decode phase is memory-bandwidth-bound — reading all weights per token is the bottleneck

L3 — Python & PyTorch Performance

Why Python is Slow

Source of Overhead	Detail
Dynamic typing	Type checked on every operation; cannot optimize at compile time
PyObject overhead	Every float = heap-allocated PyObject (28 bytes); boxing/unboxing
Interpreter eval loop	Bytecode executed one instruction at a time (ceval.c)
Reference counting	Refcount updated on every object operation
GIL	Global Interpreter Lock prevents true multi-threading
Heap allocations	Most Python objects live on heap; cache-unfriendly

✓ 1000×1000 matrix multiply: Pure Python ≈ 45s vs C+BLAS ≈ 0.05s — 900× slower

Python Execution Pipeline

Source (.py) → CST → AST → CFG → Bytecode (.pyc) → PVM

Peephole Optimizations (applied to bytecode)

- Constant Folding: $2+2 \rightarrow 4$
- Dead Code Removal: unreachable branches removed
- Short Sequence Optimization: collapse multiple LOAD_CONST
- Variable Lookup: replace global lookups with faster local ones

Stack Machine & Bytecode

Python VM = stack-based machine. Key bytecodes:

- LOAD_CONST, LOAD_FAST, LOAD_GLOBAL, LOAD_ATTR — push to stack
- BINARY_ADD, BINARY_MULTIPLY — pop 2, push result
- CALL_FUNCTION(a) — call with a args from stack
- STORE_FAST, RETURN_VALUE — write/return from stack

`d = a + b*c` → LOAD_FAST a, LOAD_FAST b, LOAD_FAST c, BINARY_MULTIPLY, BINARY_ADD, STORE_FAST d

How PyTorch Bridges Python & C++

Mechanism	How It Works	Used By
CPython C API	C code links to libpython, manipulates PyObject* directly	NumPy, SciPy
pybind11	Header-only C++ auto-generates Python bindings	PyTorch ATen (~2000 ops)
Graph Serialization	Python builds graph, hands to C++ runtime	TF tf.function, ONNX

Bytecode Capture	Intercepts Python bytecode at runtime, extracts compute graph	TorchDynamo (torch.compile)
Dynamic Codegen	Generates C++/CUDA/Triton kernels at runtime from captured graph	TorchInductor, XLA, TVM

torch.compile Stack (PyTorch 2.0)

The flagship feature of PyTorch 2.0. Zero code changes required: `model = torch.compile(model)`

Component	Role	Key Detail
TorchDynamo	Graph Acquisition	Intercepts CPython bytecode frame evaluation API; captures FX graph; handles dynamic control flow via GUARDS
AOTAutograd	Graph Lowering	Captures forward AND backward graphs ahead-of-time; enables joint optimization
TorchInductor	Graph Compilation	Default backend; generates Triton (GPU) or C++/OpenMP (CPU) kernels at runtime
Guards	Cache Validity	Shape/dtype guards ensure compiled graph matches current input; recompiles on miss

✓ torch.compile speedups on A100: +38% TIMM, +76% TorchBench, +52% HuggingFace

★ EXAM: TorchDynamo hooks into CPython's frame evaluation API — this is what enables zero-code-change compilation

Eager vs. Graph Execution

Aspect	Eager (Imperative)	Graph (Declarative)
Execution	Immediate per op	Define → Compile → Run
Debugging	Easy (intermediate values visible)	Hard (graph must execute fully)
Optimization	No global view	Operator fusion, memory reuse
Examples	PyTorch default	TF 1.x, @tf.function, JAX
Dynamic control flow	Native Python support	Limited / requires annotation

PyTorch Autograd & Computation Graphs

- Set `requires_grad=True` to track operations on a tensor
- Calling `.backward()` computes gradients via chain rule through the dynamic graph
- Gradients accumulate in `.grad` attribute
- `torch.no_grad()` context manager: no graph built — use for weight updates & inference

■ MUST call `zero_grad()` before each backward pass — gradients accumulate by default!

$$z = 2x^2 + 5x \rightarrow dz/dx = 4x + 5 \rightarrow \text{at } x=2: \text{grad} = 13$$

DataLoader Key Facts

- Abstracts batching, shuffling, and parallel loading of data
- **num_workers > 0**: spawns worker processes (bypasses GIL) for parallel data loading
- **pin_memory=True**: uses pinned (page-locked) host memory → faster H2D transfers
- **prefetch_factor**: preload batches ahead of training loop
- DataLoader does NOT update model weights

★ EXAM: Profiler shows DataLoader=420ms, GPU idle=410ms → fix the input pipeline (num_workers, pin_memory)

Python Multiprocessing & GIL

- **GIL (Global Interpreter Lock)**: prevents multiple threads from executing Python bytecode in parallel
- Python threads: **concurrency but NOT parallelism** (good for I/O-bound, bad for CPU-bound)
- **torch.multiprocessing**: extends Python multiprocessing with GPU-aware tensor sharing
- **Default start method: 'spawn'** (creates fresh process) — CUDA-safe

■ Never use 'fork' with CUDA — inherits corrupted GPU state (CUDA context is not fork-safe)

Method	CUDA Safe?	Speed	Notes
fork	UNSAFE	Fast	Inherits parent memory incl. GPU state → corruption
spawn (default)	SAFE	Slower	Creates clean new process; default for PyTorch+CUDA
forkserver	Unsafe	Moderate	Still unsafe for CUDA

L4 — CUDA Basics

CPU vs. GPU Design Philosophy

CPU (Latency Optimized)	GPU (Throughput Optimized)
Few, powerful cores (up to 60+)	Thousands of small cores (CUDA Cores)
Sophisticated branch prediction & OOO exec	Simple control — no branch prediction
Large L1/L2/L3 cache hierarchy	Small caches; large shared memory
Low latency per instruction	High throughput via massive parallelism
Best for sequential code	Best for parallel data-parallel code
Sequential $\approx 10\times$ faster	Parallel $\approx 10\text{--}100\times$ faster

✓ GPU kernel launch overhead $\approx 10\text{s}$ of microseconds. Use GPU only when parallelism > launch cost.

CUDA Thread Hierarchy

Fundamental structure: **Thread** $\subset$ **Block** $\subset$ **Grid**

Concept	Maps To	Key Properties
Thread	SP (Streaming Processor / CUDA Core)	Smallest unit; has own registers and local memory
Block (Thread Block)	SM (Streaming Multiprocessor)	Max 1024 threads; threads can share memory and sync
Grid	GPU (Device)	One grid per kernel launch; blocks execute in any order
Warp	32 consecutive threads in a block	Scheduling unit; all execute same instruction (SIMT)

★ EXAM: Warp = 32 threads. Max threads/block = 1024. Never forget these two numbers.

Global Thread Index Formula

$i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ (1D)

$\text{Row} = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$; $\text{Col} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ (2D)

Always check: **if ($i < N$)** to avoid out-of-bounds access when N is not a multiple of blockDim .

★ EXAM: COMMON BUG: using only `threadIdx.x` without `blockIdx.x*blockDim.x` → only block 0 works correctly!

CUDA Function Qualifiers

Qualifier	Executes On	Called From	Notes
<code>__global__</code>	GPU (device)	CPU (host)	Returns void; kernel launch entry point
<code>__device__</code>	GPU (device)	GPU only	Helper functions on GPU

__host__	CPU (host)	CPU only	Default; can combine with __device__
----------	------------	----------	--------------------------------------

Kernel Launch Syntax & Memory APIs

```
kernel<<>>(args);
cudaMalloc(&d_ptr, N*sizeof(float)); // Allocate on device
cudaMemcpy(d_ptr, h_ptr, N*sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(h_ptr, d_ptr, N*sizeof(float), cudaMemcpyDeviceToHost);
cudaFree(d_ptr); // Free device memory
cudaDeviceSynchronize(); // Force host to wait for GPU
```

Convention: h_ prefix = host memory; d_ prefix = device memory

CUDA Memory Model — Complete Hierarchy

Level	Location	Latency	BW	Scope	Managed By	Size
Registers	On-chip (SM)	~1 cycle	~8 TB/s	Per thread	Compiler	Thousands/SM
Shared Memory / L1	On-chip (SM)	~20 cycles	~1.5–19 TB/s	Per block	Programmer	192KB (A100), 256KB (H100)
L2 Cache	On-chip (die)	~200 cycles	—	All SMs	Hardware	40MB (A100), 50MB (H100)
Global (HBM)	Off-chip	~400–600 cycles	2 TB/s (A100), 3.35 TB/s (H100)	All SMs	Programmer	40–80 GB
Constant	Off-chip (cached)	~1 (cached)	—	All threads (read-only)	Programmer	64 KB

- ✓ Shared memory is programmer-managed L1 cache. Declare with __shared__. Must sync with __syncthreads().
- ✓ Register spilling = spilling to local memory = actually global memory = slow!

Unified Virtual Memory (UVM)

- cudaMallocManaged(&ptr, size) — single pointer accessible from both CPU and GPU
- CUDA 8+ / Pascal+: demand paging — only touched pages migrate
- cudaMemPrefetchAsync() — restores performance by pre-migrating data

■ UVM is for programming convenience, NOT performance. Expert manual cudaMemcpy is always faster.

Warp Divergence

All threads in a warp execute the SAME instruction (SIMT). If threads take different branches:

- Both paths are executed **serially** — one path masked/inactive per pass
- Reduces effective parallelism by the divergence factor
- **Solution:** Organize data so threads in same warp follow the same branch

★ EXAM: Warp divergence = performance killer. If/else inside a warp = serialized execution.

__syncthreads() and Synchronization

- **__syncthreads()**: barrier for threads within same block. All must reach before any proceed.
- Use AFTER writing shared memory, BEFORE reading it (to prevent race conditions)
- **cudaDeviceSynchronize()**: host waits for ALL GPU work to complete
- Kernel launches are ASYNCHRONOUS — host continues immediately after launch
- **cudaMemcpy()** is SYNCHRONOUS by default — host waits

SM Occupancy

Occupancy = Active Warps per SM / Max Warps per SM. Goal: maximize occupancy to hide latency.

$$\text{Warps per SM} = (\text{threads_per_block} / 32) \times \text{blocks_assigned_to_SM}$$

Example (Fermi, max 1536 threads/SM, max 8 blocks):

Block Config	Threads/Block	Warps/Block	Max Blocks	Total Warps	Occupancy
8×8	64	2	8	16	33% (LOW)
16×16	256	8	6	48	100% (OPTIMAL)
32×32	1024	32	1	32	67%

★ EXAM: 16×16 block (256 threads) = optimal occupancy for Fermi. Always think in multiples of 32.

L5 — Advanced CUDA

GPU Architecture Comparison

Spec	Tesla V100 (Volta)	A100 (Ampere)	H100 (Hopper)
CUDA Cores	5,120	6,912	18,432
Tensor Cores	640 (1st/2nd gen)	432 (3rd gen)	640 (4th gen + FP8)
SMs	80	108	132
Peak FP32	15.7 TFLOPS	19.5 TFLOPS	60 TFLOPS
Peak TF16 (Tensor)	125 TFLOPS	312 TFLOPS	1,000 TFLOPS
HBM BW	~900 GB/s	~2 TB/s (HBM2e)	~3.35 TB/s (HBM3)
L2 Cache	6 MB	40 MB	50 MB
Shared Mem/SM	96 KB	192 KB	256 KB
NVLink BW	300 GB/s	600 GB/s	900 GB/s

Tensor Cores

Specialized hardware for mixed-precision matrix multiply-accumulate: $D = A \times B + C$

- A, B in FP16/BF16/INT8/FP8 → C, D in FP16 or FP32 (accumulate in higher precision)
- Perform 16×16 matrix operations per cycle — 4–16× faster than CUDA cores for GEMM
- **3rd Gen (A100)**: TF32, BF16, FP64, 2:4 structured sparsity (2× throughput)
- **4th Gen (H100)**: Adds FP8 (E4M3 forward, E5M2 backward); Transformer Memory Accelerator (TMA)

✓ Tensor Cores power ALL modern LLM training. FP8 on H100 = 2× more FLOPS than BF16.

CUDA Streams

A stream = an ordered queue of GPU work. Host places work, device executes when resources free.

- Operations in the **same stream**: ORDERED (FIFO), cannot overlap
- Operations in **different streams**: UNORDERED, CAN overlap
- Default stream: all operations go here if no stream specified

```
cudaStreamCreate(&stream1);
cudaMemcpyAsync(d_a, h_a, size, cudaMemcpyHostToDevice, stream1);
my_kernel<<>>(d_a);
cudaStreamSynchronize(stream1);
```

✓ Streams enable overlapping H2D transfers with kernel execution → hides memory latency

Pinned (Page-Locked) Memory

- Allocated with `cudaMallocHost()` — memory cannot be paged out by OS
- Required for `cudaMemcpyAsync()` to work correctly
- `cudaMemcpy()` is faster with pinned source/dest (DMA can transfer directly)

- Downside: limits virtual memory available to OS; use judiciously

Memory Coalescing (Critical for Global Memory Performance)

When threads in a warp access consecutive memory addresses, HW coalesces into one transaction.

Access Pattern	Transactions	Performance
Stride-1 (consecutive)	1 transaction	Optimal — all threads in one 128-byte line
Stride-2	2 transactions	Half efficiency
Stride-32 (worst)	32 transactions	Terrible — one transaction per thread

★ EXAM: Pattern A (stride-1) coalesces to 1 transaction; Pattern B (stride-32) = 32 transactions = 32× slower

Shared Memory Bank Conflicts

- Shared memory divided into 32 banks (4-byte granularity)
- Consecutive 4-byte words → consecutive banks
- **Bank conflict:** 2+ threads in warp access different addresses in same bank → serialized
- **No conflict:** All threads access different banks (stride-1 for 32 threads = no conflict)
- **Broadcast:** All threads access same address in same bank → no conflict (1 read, broadcast)
- Fix: use padding (e.g., `__shared__ float tile[32][33]`) to shift columns across banks

Tiled Matrix Multiplication

Key optimization: load tiles of A and B into shared memory, reuse for all C elements in tile.

- Each element of global memory loaded only once per tile instead of once per multiply
- Tile size $TILE \times TILE$; larger tile = more reuse = higher arithmetic intensity
- Requires `__syncthreads()` after loading tile (before compute) and after compute (before next tile)

✓ Tiled matmul: $AI = TILE/2 \text{ FLOP/byte}$ vs. naive $AI \approx 1/4 \text{ FLOP/byte}$ → moves toward compute-bound

Compute Capability Reference

Compute Capability	Architecture	Key Feature
7.0	Volta	Tensor Cores introduced; V100
7.5	Turing	INT8 Tensor Cores; RTX 20xx
8.0	Ampere	TF32, BF16, FP64 Tensor Cores; 2:4 sparsity; A100
9.0	Hopper	FP8 Tensor Cores; TMA; Transformer Engine; H100

CUDA Event API for Timing

```
cudaEvent_t start, stop; float ms;
cudaEventCreate(&start); cudaEventCreate(&stop);
cudaEventRecord(start, 0);
my_kernel<>>(args);
cudaEventRecord(stop, 0);
```



```
cudaEventSynchronize(stop); // Wait for kernel  
cudaEventElapsedTime(&ms, start, stop); // ms in milliseconds
```

■ **Standard CPU timers (`clock()`, `gettimeofday()`) CANNOT measure GPU timing correctly — always use CUDA Events**

L6 — Distributed Deep Learning (DDL)

Why Distributed Training

- LLaMA 3 405B at fp16 = **810 GB** → single H100 (80 GB) cannot hold even the weights
- GPT-3 175B with Adam optimizer ≈ **2.8 TB** (16 bytes/param)
- Modern LLM training: 1,000 – 100,000+ GPUs running for weeks

✓ TSM video model: 50 hours → 14 minutes with 1,536 GPUs (211× speedup)

Parallelism Strategies

Strategy	How It Works	Best For	Key Challenge
Data Parallelism	Same model on each GPU; different data batches; AllReduce gradients	Most cases; batch size can grow	Communication overhead of AllReduce
Model / Tensor Parallelism	Split weight matrices within a layer across GPUs	Huge models; single layer too large	High BW needed (NVLink); all-to-all per layer
Pipeline Parallelism	Different layers on different GPUs; microbatches fill pipeline	Extremely deep models	Pipeline bubbles at start/end
ZeRO / FSDP	Shard optimizer states, gradients, AND parameters across GPUs	Memory-constrained training	AllGather + Reduce-Scatter overhead
4D / Hybrid	DP + TP + PP + SP combined	LLM pre-training at 1000s of GPUs	Complex configuration

Communication Collective Primitives

★ EXAM: Know what each collective does — exam question on which to use for gradient averaging

Primitive	What It Does	DL Use Case
Broadcast	Root → all: same data to everyone	Initialize model parameters on all workers
Reduce	All → root: combine (sum/max/min) from all	Gradient accumulation at parameter server
AllReduce	All → all: combine AND distribute to everyone	Gradient averaging in data parallelism (THE key collective)
Reduce-Scatter	Each rank gets partial result (its shard)	ZeRO gradient sharding
AllGather	Each has partial; all get full data	ZeRO parameter reconstruction before forward pass
AllToAll	Each rank sends DIFFERENT data to each other	Expert routing in Mixture-of-Experts

★ EXAM: AllReduce = most important collective in deep learning. Used for gradient synchronization in data parallelism.

Ring AllReduce Algorithm

Bandwidth-optimal collective used by NCCL. Two phases:

- **Phase 1 — Scatter-Reduce:** Each of $N-1$ iterations: send to right neighbor, receive from left; each node accumulates partial sum
- **Phase 2 — AllGather:** Each of $N-1$ iterations: send completed chunk right; all nodes receive full result

Communication cost per GPU = $2N(P-1)/P \approx 2N$ for large P

Parameter Server cost = $2N(P-1)$ [grows linearly with P – bottleneck!]

✓ Ring AllReduce is nearly bandwidth-independent of #GPUs. PS becomes a bottleneck at scale.

Synchronous vs. Asynchronous SGD

Aspect	Synchronous (Sync-SGD)	Asynchronous (ASGD)
Update timing	All workers must finish before update	Workers update independently
Gradient staleness	None — all fresh	Yes — stale gradients possible
Determinism	Fully deterministic	Non-deterministic
Straggler sensitivity	High — slow worker stalls all	Low — slow workers don't block others
Convergence	Mathematically equivalent to SGD	May diverge with high staleness
Used in	PyTorch DDP/FSDP (default)	Hogwild, some RL scenarios

★ EXAM: Slow straggler stalls synchronous data-parallel SGD = TRUE exam answer

ZeRO (Zero Redundancy Optimizer)

Shards model state across GPUs while preserving data-parallel semantics. Three stages:

Stage	What Is Sharded	Memory/GPU (N=64)	Max Model Size
Baseline (DP)	Nothing — full replica	~120 GB (all)	5B params
ZeRO-1	Optimizer states only	~31 GB (~4× reduction)	~19B params
ZeRO-2	Optimizer states + Gradients	~17 GB (~8× reduction)	~36B params
ZeRO-3 / FSDP	Optimizer + Gradients + Parameters	~2 GB (~64× reduction)	~320B params

✓ ZeRO-3 is implemented in PyTorch as FullyShardedDataParallel (FSDP). Use FSDP2 (PyTorch 2.4+) for best performance.

FSDP Forward/Backward Flow

- AllGather parameters → Forward pass → Free full weights → AllGather parameters → Backward pass → Reduce-Scatter gradients → Free full weights → Update local shard

Pipeline Parallelism

Split model layers across GPUs. Microbatches pipeline through stages.

Variant	Key Feature	Memory	Bubble
Naive	One microbatch at a time	Low	High (~1 - 1/m)
GPipe	M microbatches; all fwd then all bwd	O(M) activations	Moderate
1F1B (Interleaved)	Prioritize backward; release activations ASAP	Lower	~Same as GPipe
Zero-Bubble (ZB-H1)	Defer W-grad to fill bubbles	Same as 1F1B	~50% less
Zero-Bubble (ZB-H2)	Near-zero bubbles	Slightly higher	~Near zero

Communication Libraries

Library	Type	Used By	Notes
NCCL	GPU-GPU collectives	PyTorch DDP/FSDP2, DeepSpeed, Megatron	Topology-aware; NVLink/IB; MOST POPULAR
Gloo	CPU-based (Meta)	PyTorch CPU fallback	No NCCL available? Gloo is default
MPI	General HPC	Horovod, academic clusters	CUDA-aware MPI for direct GPU transfers
UCX	Low-level transport	HPC, RDMA, DeepSpeed	Ultra-low latency GPU-to-GPU
SHARP	In-network reduction	InfiniBand Quantum-2	Hardware-accelerated AllReduce in switch

Scale-Up vs. Scale-Out

- **Scale-up (intra-node):** NVLink/NVSwitch → 900 GB/s (DGX H100) — fastest path
- **Scale-out (inter-node):** InfiniBand NDR → up to 400 Gb/s — slower but needed for large clusters
- **DDP (DistributedDataParallel):** Full model on each GPU; AllReduce gradients
- **FSDP (FullyShardedDataParallel):** ZeRO-3; shards everything; handles 100B+ models

L7 — Neural Network Quantization

Why Quantize

- Reduce model memory: FP32→INT8 = 4× smaller; FP32→FP16 = 2× smaller
- Faster inference: INT8 matmul ~4× faster than FP32 on specialized hardware
- Lower energy: DRAM access = 640 pJ; INT8 mult = 0.2 pJ vs FP32 mult = 3.7 pJ (18.5× less)
- Enable edge deployment: smaller models fit on mobile/embedded devices

✓ DeepSeek-R1 671B: 720 GB original → 131 GB quantized = 5.5× reduction

Floating Point Formats

Format	Sign	Exponent	Mantissa	Total	Key Property
FP32 (IEEE)	1	8	23	32	Full precision; baseline
FP16 (IEEE)	1	5	10	16	2× smaller; narrower range → overflow risk
BF16 (Google)	1	8	7	16	Same range as FP32; drop-in for training
TF32 (NVIDIA)	1	8	10	19	FP32 range + FP16 precision; A100 default
FP8 E4M3	1	4	3	8	H100 forward pass (weights/activations)
FP8 E5M2	1	5	2	8	H100 backward pass (gradients — needs more range)
INT8	—	—	—	8	Fixed-point; symmetric or asymmetric
INT4	—	—	—	4	Aggressive; needs careful calibration

★ EXAM: BF16 has same exponent bits as FP32 → same range → safe drop-in for training. FP16 can overflow.

Linear (Uniform) Quantization — Core Math

$r = S(q - Z)$ [dequantization: int → float]

$q = \text{clamp}(\text{round}(r/S) + Z, q_{\min}, q_{\max})$ [quantization: float → int]

Where: r = real float value, q = quantized integer, S = scale (step size), Z = zero-point

Scale and Zero-Point Formulas

$S = (r_{\max} - r_{\min}) / (q_{\max} - q_{\min})$

$Z = \text{round}(q_{\min} - r_{\min}/S)$

Example (INT8, $r_{\min}=-1.8$, $r_{\max}=2.5$): $S = (2.5+1.8)/255 \approx 0.01686$; $Z = \text{round}(-128+1.8/0.01686) = -21$

Symmetric vs. Asymmetric Quantization

Mode	Zero-Point	Range	Best For
Symmetric	$Z = 0$	$[- r_{\max} , r_{\max}]$ — wasted if skewed	Weights (usually symmetric about 0)

Asymmetric	Z ≠ 0	[r_min, r_max] — full utilization	Activations after ReLU (all positive)
------------	-------	-----------------------------------	---------------------------------------

✓ Best practice: Symmetric weights + Asymmetric activations = hardware-efficient combination

Per-Tensor vs. Per-Channel Quantization

- **Per-Tensor:** Single S, Z for entire tensor. Fastest, most hardware compatible.
- **Per-Channel:** Separate S, Z per output channel. Better accuracy. No special HW needed for weights.

PTQ vs. QAT

Aspect	PTQ (Post-Training Quantization)	QAT (Quantization-Aware Training)
Training needed	No (or calibration data only)	Yes — simulate quantization during training
Accuracy	Good for 8-bit; degrades at 4-bit	Better, especially at low bit-widths
Speed of setup	Fast	Slow (requires full training run)
How it works	Quantize trained FP32 model directly	Insert fake-quantization nodes; network adapts
Use case	Production inference; edge deployment	When PTQ accuracy is insufficient

Static vs. Dynamic Quantization

- **Static (Offline PTQ):** Calibration pass determines fixed scale/zero-point for activations. Fast at inference.
- **Dynamic (Runtime):** Quantize activations per-inference based on observed min/max. Slower but more accurate.

FP8 Training Details (H100)

Two FP8 formats optimized for different roles:

Format	Exponent	Mantissa	Has Inf?	Best For
FP8 E4M3	4	3	No (NaN only)	Forward pass: weights & activations (more precision)
FP8 E5M2	5	2	Yes	Backward pass: gradients (needs higher dynamic range)

✓ FP8 on H100: 2× more FLOPS vs BF16. Key for training trillion-parameter models efficiently.

Simulating Quantization

- Insert quantizer nodes into compute graph to simulate fixed-point on FP32 hardware (CPU/GPU)
- Forward: quantize → dequantize (FP32 → round to INT8 levels → FP32 approximation)
- Backward: Straight-Through Estimator (STE) — pass gradients through quantizer as if identity
- Benefits: GPU acceleration, no dedicated kernel, test various bit-widths quickly

L8 — Neural Network Pruning & Sparsity

Why Prune

- Neural networks are massively over-parameterized — >90% of weights can be removed with minimal accuracy loss
- **NVIDIA MLPerf Llama 2 70B**: 2.5× speedup, 99% accuracy retained via depth+width pruning
- Reduces storage, memory bandwidth, and compute requirements

✓ Pruning 90% of AlexNet weights: 9× fewer parameters, 3× fewer MACs, nearly same accuracy

Pruning Granularity Spectrum

Granularity	Description	HW Acceleration	Compression
Fine-grained (Unstructured)	Any individual weight	Custom HW only (EIE accelerator)	Highest
N:M Pattern (2:4)	2 zeros per 4 consecutive weights	NVIDIA Sparse Tensor Cores (A100+): 2×	High
Vector-level	Entire rows/columns	Standard GEMM	Moderate
Head Pruning	Remove entire attention heads	Standard GEMM (fewer heads)	Moderate
Width Pruning	Reduce FFN/hidden dims	Standard GEMM (smaller matrices)	Moderate
Depth Pruning	Remove entire transformer layers	Trivial — fewer forward passes	Lower

★ EXAM: Fine-grained = more compression but GPU-unfriendly. Structured (channel/depth) = GPU-friendly, less compression.

2:4 Structured Sparsity (NVIDIA Ampere+)

Exactly 2 zeros in every 4 consecutive weights = 50% sparsity. Natively accelerated by Sparse Tensor Cores.

- Compressed format: non-zero values + 2-bit indices per 4 elements
- A100: up to **2× peak throughput** for sparse matrix multiplication

Model	Dense FP16	Sparse FP16 (2:4)	Accuracy Drop
ResNet-50 ImageNet Top-1	76.1%	76.2%	None (slight gain!)
BERT-Large SQuAD F1	91.9	91.9	None
FairSeq Transformer BLEU	28.2	28.5	None (slight gain!)

Pruning Criteria

Magnitude-Based (Most Common)

Importance = $|W|$ (element-wise for unstructured)

$$\text{Importance} = ||W^{\wedge}(S)||_p \text{ (group-wise for structured)} - \text{L1 or L2 norm}$$

Wanda (Weights AND Activations)

$$\text{Score} = |W_{\{i,j\}}| \times ||X_j||_2 \text{ (weight magnitude} \times \text{input activation norm)}$$

'A small weight times a huge activation is not small.' Beats plain magnitude at all sparsity levels.

SparseGPT (Hessian-Based)

$$\text{Saliency} = W_{\{t,j\}}^2 / [H^{-1}]_{\{jj\}} \text{ where } H = 2XX^T + \lambda I$$

$$\text{Weight update} = \Delta W_{\{t,k\}} = -(W_{\{t,j\}}/[H^{-1}]_{\{jj\}}) \times [H^{-1}]_{\{jk\}}$$

One-shot unstructured pruning using approximate Hessian inverse. No retraining needed. Prunes OPT-175B in ~4 hours on 1 GPU.

Iterative Pruning Pipeline

Step	Action	Notes
1	Train Dense Model	Full model to convergence
2	Compute Importance	Magnitude, Wanda, or SparseGPT score
3	Prune Weights	Remove lowest-importance at target granularity
4	Fine-Tune (Recover)	1/10–1/100 of original LR; 1–3 epochs
5	Iterate or Deploy	Repeat steps 2–4 for higher sparsity

✓ Pruning only → accuracy drops. Pruning + fine-tuning → maintains accuracy to ~90% sparsity. Iterative = best.

Transformer Pruning Summary

Component	Pruning Method	Effect
Individual weights	Unstructured / 2:4 sparsity	Max compression; needs sparse HW or Sparse Tensor Cores
Attention heads	Head pruning	Remove d_head columns from W_Q/K/V/O; reduces attention cost
FFN dims	Width pruning	Reduce 4d→2d intermediate; 50% FFN cost reduction
Full layers	Depth pruning	Remove entire transformer blocks; NVIDIA Llama 2: 80→32 layers

✓ NVIDIA Llama 2 70B: depth pruning (80→32 layers) + width pruning → 2.5× speedup, 99% accuracy retained

Modern LLM Pruning Methods

Method	Type	Key Idea	No Retrain?
SparseGPT	Unstructured	Hessian-based weight updates after pruning; compensates errors	Yes
Wanda	Unstructured	Score = $ w \times x $; activations matter as much as weights	Yes

Minitron (NVIDIA)	Structured depth+width	Prune + KD; 40× fewer tokens than training from scratch	No (KD)
Sheared-LLaMA	Joint depth+width+head	Learned masks; outperforms same-size from-scratch models	No

L9 — Efficient Transformers & FlashAttention

Self-Attention Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

$Q = XW_Q$ (what am I looking for?), $K = XW_K$ (what do I have?), $V = XW_V$ (what to extract?)

- $\text{Score}_{\{ij\}} = Q_i \cdot K_j^T / \sqrt{d_k}$ (scaled dot-product)
- $\alpha_{\{ij\}} = \text{softmax}(\text{Score}_{\{ij\}})$ (attention weights)
- $\text{Output}_i = \sum_j \alpha_{\{ij\}} \cdot V_j$ (weighted sum of values)
- **Causal (GPT)**: attends only to preceding positions; **Non-causal (BERT)**: bidirectional
- Standard complexity: **$O(N^2)$ time and memory** — $N \times N$ attention matrix is the bottleneck

Why Standard Attention is Memory-Bound

Standard attention requires materializing the full $N \times N$ attention matrix in HBM:

- Step 1: $QK^T \rightarrow$ write $N \times N$ matrix S to HBM
- Step 2: read S from HBM $\rightarrow$ softmax $\rightarrow$ write P to HBM
- Step 3: read P, V from HBM $\rightarrow PV \rightarrow$ write O to HBM

Each step = expensive HBM read+write. HBM (1.5 TB/s) is 13× slower than SRAM (19 TB/s).

★ **EXAM: Standard attention OOMs for long sequences because $N \times N$ matrix in HBM scales quadratically**

FlashAttention — Core Insight

✓ **IO-aware attention: minimize HBM reads/writes, not FLOPs. Same $O(N^2)$ FLOPs, much less HBM traffic.**

Two Key Techniques

1. **Tiling**: Process attention in small tiles that fit in SRAM. Never materialize full $N \times N$ matrix.
2. **Online Softmax**: Compute softmax incrementally block-by-block using running statistics.

FlashAttention Algorithm — Key Steps

Inputs: $Q, K, V \in \mathbb{R}^{(N \times d)}$ in HBM; SRAM of size M

$$\text{Block sizes: } B_c = \lfloor M/4d \rfloor, B_r = \min(\lfloor M/4d \rfloor, d)$$

- **Outer loop**: iterate over K, V blocks ($T_c = \lceil N/B_c \rceil$ iterations)
- Load K_j, V_j into SRAM
- **Inner loop**: iterate over Q blocks ($T_r = \lceil N/B_r \rceil$ iterations)
- Load Q_i, O_i, m_i into SRAM
- Compute $S_{ij} = Q_i \cdot K_j^T$ on-chip (small tile, not full $N \times N$)
- Compute local max $m_{ij} = \text{rowmax}(S_{ij})$ and local sum $\tilde{m}_{ij} = \text{rowsum}(\exp(S_{ij} - m_{ij}))$
- Update running stats: $m_{\text{new}} = \max(m_{\text{old}}, m_{ij})$; $\tilde{m}_{\text{new}} = e^{(m_{\text{old}} - m_{\text{new}})} \cdot \tilde{m}_{\text{old}} + e^{(m_{ij} - m_{\text{new}})} \cdot \tilde{m}_{ij}$
- Rescale and accumulate O_i with new block's contribution; write back to HBM

Online Softmax Statistics

$$m(x) = \max_i(x_i) \text{ [running maximum]}$$

$$\blacksquare(x) = \sum_i \exp(x_i - m(x)) \text{ [running sum for denominator]}$$

$$\text{softmax}(x) = f(x) / \blacksquare(x) \text{ where } f(x) = [\exp(x_1 - m), \dots, \exp(x_B - m)]$$

FlashAttention Performance

Metric	Standard Attention	FlashAttention	Improvement
HBM R/W (GPT-2 medium)	40.3 GB	4.4 GB	~9× less
Runtime (GPT-2 medium)	41.7 ms	7.3 ms	~5.7× faster
GFLOPs (GPT-2 medium)	66.6	75.2	13% more (recomputation)
Memory complexity	$O(N^2)$	$O(N) \leftarrow$ linear!	10–20× less
Wall-clock speedup	—	2–4× (up to 6×)	With masking+dropout: 4×
BERT training (MLPerf)	20.0 ± 1.5 min	17.4 ± 1.4 min	~15% faster

★ EXAM: FlashAttention has same $O(N^2)$ FLOPs as standard attention — it only reduces HBM I/O. This is a common trick question!

Backward Pass: Recomputation

- FlashAttention does NOT store the $N \times N$ attention matrix P from the forward pass
- In backward, recompute attention from Q, K, V using saved softmax statistics (m and $\blacksquare$)
- Tradeoff: ~13% more FLOPs vs. 10–20× less memory
- This is why FlashAttention backward stores O (output) and $(m, \blacksquare)$ — but not P

Kernel Fusion

A **kernel** = a GPU operation. **Fusion** = combining multiple operations into one kernel.

- Standard: QK^T (kernel 1) $\rightarrow$ softmax (kernel 2) $\rightarrow$ dropout (kernel 3) $\rightarrow$ V (kernel 4)
- FlashAttention: single fused kernel — load from HBM once, all ops in SRAM, write once
- Eliminates repeated HBM round-trips for each intermediate result

✓ Kernel fusion is the primary implementation technique enabling FlashAttention's speedup

vLLM & PagedAttention (Bonus)

- Traditional LLM inference: pre-allocates max-length contiguous KV cache $\rightarrow$ 60–80% GPU memory wasted
- PagedAttention**: Splits KV cache into fixed-size blocks (e.g., 16 tokens), stored non-contiguously
- Block Table maps logical $\rightarrow$ physical GPU addresses; allocate on-demand
- Continuous Batching**: finished requests immediately replaced; no GPU idle 'bubbles'

✓ vLLM achieves 2–4× higher throughput vs static batching

L10 — Knowledge Distillation

Core Concept

Train a small **student model** to replicate behavior of a large **teacher model**.

- **Key observation:** Large networks are easier to train; small networks are easier to deploy
- **DistilBERT (2019):** 40% smaller than BERT, retains 97% of language understanding

Three Types of Knowledge

Type	Source	How Transferred
Response-Based	Last layer logits (output)	KL divergence between soft teacher/student distributions
Feature-Based	Intermediate layer activations	MSE or KL between teacher/student feature maps
Relation-Based	Relationships between feature maps	FSP matrix (inner products between features from two layers)

Hard vs. Soft Labels & Temperature

Hard labels: One-hot [0,0,0,1,0,...] — no inter-class similarity information

Soft labels: Teacher's probability distribution — encodes 'dark knowledge' (similarity between classes)

$$\text{Softmax with temperature } T: q_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$$

- $T = 1$: standard peaked softmax
- $T > 1$: smoother distribution → more dark knowledge exposed → more informative for student
- $T = 5, 7, 10$: progressively flatten; expose class similarities more clearly

Loss Functions

KL Divergence

$$D_{KL}(P||Q) = \sum_x p(x) \ln(p(x)/q(x))$$

Non-symmetric. Measures relative entropy between distributions. Used for distillation loss.

Total Distillation Loss

$$L_{total} = \alpha \cdot L_{student} + (1-\alpha) \cdot L_{distill}$$

- $L_{student}$ = cross-entropy with hard (one-hot) labels
- $L_{distill}$ = KL divergence between teacher and student soft distributions (at temperature T)
- $\alpha \in [0,1]$: higher α → more hard labels; lower α → more soft knowledge

FitNets (Feature-Based KD)

Train deeper and thinner students using teacher's intermediate layers as 'hints'.

- **Hint layer:** teacher intermediate layer whose output student learns to predict
- **Guided layer:** student layer that learns from hint
- Stage 1: Hints training — match intermediate features via MSE

- Stage 2: Full KD — train entire student with soft labels

✓ FitNet 1: 11 layers, 250K params → 13× speedup, 36× compression vs teacher (5 layers, 9M params)

Training Modes

Mode	When	How
Offline (most common)	Pre-trained teacher available	Teacher frozen; student trains from scratch with teacher supervision
Online	No pre-trained teacher	Teacher and student trained simultaneously in end-to-end fashion
Self-Distillation	No teacher needed	Same model — deeper layers supervise shallower layers; or later epoch supervises earlier

NVIDIA Minitron: Production KD

Prune large model then use KD to recover accuracy. 40× cheaper than training from scratch.

Model	Params	Method	MMLU
Llama-3.1-Minitron-4B (width)	4B	Width prune + distill	60.5
Llama-3.1-Minitron-4B (depth)	4B	Depth prune + distill	58.7
MN-Minitron-8B	8B	Width prune + distill	69.5

Modern LLM Architecture & Serving

Transformer Architecture Overview

Component	Formula/Detail	Notes
Self-Attention	$Q = XW_Q, K = XW_K, V = XW_V;$ $\text{softmax}(QK^T/\sqrt{d_k})V$	Multi-head: H independent heads, concat
FFN	$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$	~2/3 of transformer FLOPs
Layer Norm	$(x - \mu)/\sigma \times \gamma + \beta$	Pre-norm (modern) vs post-norm
Positional Encoding	RoPE (rotary, modern) or learned	RoPE enables longer context generalization
KV Cache	Cache K, V from all previous tokens	Size: 64 KB/token (8B model), 160 KB/token (70B)

LLM Inference: Prefill vs. Decode

Phase	Description	Bottleneck
Prefill	Process all input tokens; populate KV cache	Compute-bound (large parallel matmuls)
Decode	Generate one token at a time; read full KV cache	Memory-bandwidth-bound (read all weights per token)

★ EXAM: Decode phase = memory-bandwidth-bound. Each generated token requires reading ALL model weights.

Mixture of Experts (MoE)

- Replace dense FFN with N experts; router activates top-k per token
- DeepSeek-V3: 671B total params, 37B active per token (top-8 of 256 experts)
- Training cost: ~\$5.5M — fraction of dense equivalent (GPT-4 est. \$100M+)
- Communication: AllToAll for routing tokens to experts across GPUs

✓ MoE = 'pruning by design' — only 5–15% of params active per token

Speculative Decoding

Small draft model proposes K tokens; large verifier model checks all K in one forward pass.

- If all K tokens accepted → K× more throughput than sequential decoding
- If rejected at token j → discard tokens j+1...K, continue from j
- **Speedup: 2–4× in practice** when draft model is well-matched to verifier

✓ Speculative decoding exploits that the verifier can check K tokens as fast as generating 1

LoRA (Low-Rank Adaptation)

$$W' = W + \Delta W = W + A \cdot B \text{ where } A \in \mathbb{R}^{(d \times r)}, B \in \mathbb{R}^{(r \times k)}, r \ll \min(d, k)$$

- Freeze pre-trained weights W; only train low-rank matrices A and B

- Typical rank $r = 4, 8, 16 \rightarrow$ massive parameter reduction vs full fine-tuning
- At inference: merge ΔW into $W \rightarrow$ no added latency

✓ LoRA: same quality as full fine-tuning with 10,000× fewer trainable parameters for LLMs

Combined Optimization Stack for LLM Deployment

Technique	Benefit	When to Apply
2:4 Structured Sparsity	2× throughput on Ampere+ Tensor Cores	Inference + training
FP8 (H100)	2× more FLOPS vs BF16	H100 training/inference
INT8 / INT4 PTQ	4–8× memory reduction	Edge/cloud inference
FlashAttention	5× faster attention, 10× less memory	Always (use by default)
Depth Pruning	Remove redundant middle layers	LLM compression
Width Pruning	Reduce FFN/hidden dims	LLM compression
Knowledge Distillation	Match quality at 40× less training cost	After pruning
vLLM / PagedAttention	2–4× throughput for serving	Production inference
Speculative Decoding	2–4× decode speedup	Low-latency interactive serving

PRACTICE QUESTIONS & ANSWERS

★ EXAM: These are representative of actual exam question types. Study all answers carefully.

Q1. What is the CUDA warp size?

Answer: B. 32 threads. Warps are the scheduling unit; all 32 execute the same instruction (SIMT).

Q2. Why does tiled matrix multiplication with shared memory improve performance?

Answer: C. Each element loaded from global memory is reused by many threads within the block, amortizing the HBM cost and increasing arithmetic intensity.

Q3. What does FlashAttention do differently from standard attention?

Answer: B. It performs the same $O(N^2)$ FLOPs as standard attention but reduces HBM I/O by tiling and online softmax, keeping intermediates in SRAM. It does NOT reduce FLOPs.

Q4. Which collective is bandwidth-optimal for gradient synchronization (used by NCCL on rings)?

Answer: B. Ring AllReduce. Cost = $2N(P-1)/P \approx 2N$ per GPU — nearly independent of P . Parameter Server = $2N(P-1)$ — grows with P .

Q5. The decode phase of autoregressive LLM inference is:

Answer: B. Memory-bandwidth-bound, because each generated token requires reading all model weights once. Arithmetic intensity is very low during decode.

Q6. Which PyTorch 2.0 component hooks into CPython's frame evaluation API?

Answer: C. TorchDynamo. It intercepts Python bytecode at the frame level to capture FX graphs without source transformation.

Q7. What is the lowest-latency memory available to a CUDA thread?

Answer: D. Registers. On-chip, per-thread, ~1 cycle latency. Followed by: Shared Memory (~20 cycles) → L2 (~200 cycles) → HBM (~400-600 cycles).

Q8. DDP training, fixed global batch, 1→8 GPUs. What type of scaling?

Answer: B. Strong scaling — same total work (fixed global batch), more GPUs → less time. Efficiency = $T_{1P} / (P \times T_{PP})$.

Q9. What is a key property of eager (imperative) computation in PyTorch?

Answer: B. Operations are executed immediately as Python statements run. No need to build a full graph first; intermediate values are accessible immediately.

Q10. Which statements about CUDA shared memory are TRUE? (select all)

Answer: A (scoped to single thread block), C (divided into 32 banks, conflicts possible), D (faster than global memory). NOT B — shared memory does NOT persist across kernel launches.

Q11. Which optimizations does torch.compile/TorchInductor apply?

Answer: A (kernel fusion), B (Triton code generation), D (graph-level dead code elimination). NOT C — torch.compile does not automatically set up distributed training.

Q12. True or False: FlashAttention reduces the FLOP count below $O(N^2)$.

Answer: FALSE. FlashAttention has the SAME $O(N^2)$ FLOPs as standard attention. It only reduces HBM I/O (memory traffic), not computation.

Q13. True or False: A slow straggler will stall synchronous data-parallel SGD.

Answer: TRUE. Synchronous SGD requires ALL workers to finish before AllReduce and update. One slow GPU delays the entire batch.

Q14. CUDA vec_add bug: kernel uses `int i = threadIdx.x` without `blockIdx.x*blockDim.x`. What's wrong?

Answer: B. Only block 0 computes correct results. Threads in other blocks have incorrect global indices (they all compute index $0 - \text{blockDim.x} - 1$). Fix: `i = blockIdx.x * blockDim.x + threadIdx.x`.

Q15. Memory access pattern A (stride-1, consecutive) vs pattern B (stride-32). Which is faster?

Answer: B $\rightarrow$ A is faster. Stride-1 coalesces all 32 warp threads into 1 HBM transaction (128-byte cache line). Stride-32 issues 32 separate transactions — $32\times$ more HBM traffic.

Q16. Profiler: DataLoader=420ms, GPU idle=410ms, forward=38ms. What to fix?

Answer: C. Fix the input pipeline: increase `num_workers` (more worker processes), set `pin_memory=True`, use `prefetch_factor`. GPU is starved waiting for data.

Q17. DDP, fixed global batch, $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ GPUs. Efficiency at 8 GPUs if $T_{\text{GPU}}=400\text{ms}$, $T_{\text{Net}}=70\text{ms}$?

Answer: B. Strong scaling. Efficiency = $T_{\text{GPU}} / (P \times T_{\text{Net}}) = 400 / (8 \times 70) = 400 / 560 \approx 0.71 = 71\%$.

Q18. GPU shows 12% utilization, CPU shows 100%. What's the bottleneck?

Answer: B. Classic CPU-bound dataloader bottleneck. CPU cannot feed data fast enough $\rightarrow$ GPU waits. Fix: more DataLoader workers, prefetch, pin_memory.

Q19. Standard attention runs OOM for long sequences. Why?

Answer: B. $Q@K.T$ materializes a dense $N \times N$ attention matrix in HBM. Memory scales with N^2 — for $N=32\text{K}$ tokens, this is $32\text{K} \times 32\text{K} = 1$ billion elements = ~ 4 GB for FP32.

Q20. Why does `.item()` inside a training loop make it $67\times$ slower?

Answer: B. `.item()` forces host-device synchronization on every call — it waits for the GPU to finish computing that scalar before returning. This serializes GPU launches instead of pipelining them.

QUICK REFERENCE — FORMULAS & CONSTANTS

All Key Formulas

Amdahl's Law

$$S = 1 / [(1-p) + p/s]$$

CUDA Global Index (1D)

$$i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

CUDA Global Index (2D)

$$\text{row} = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y} ; \text{col} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

FlashAttention block sizes

$$B_c = \lfloor M/4d \rfloor ; B_r = \min(\lfloor M/4d \rfloor, d)$$

FlashAttention HBM I/O

$$O(N^2 \cdot d^2 / M) \text{ where } M = \text{SRAM size}, d = \text{head dim}$$

Quantization (float→int)

$$q = \text{clamp}(\text{round}(r/S) + Z, q_{\min}, q_{\max})$$

Dequantization (int→float)

$$r = S(q - Z)$$

Scale S

$$S = (r_{\max} - r_{\min}) / (q_{\max} - q_{\min})$$

Zero-point Z

$$Z = \text{round}(q_{\min} - r_{\min}/S)$$

Wanda pruning score

$$S_{\{i,j\}} = |W_{\{i,j\}}| \times ||X_j||_2$$

SparseGPT saliency

$$\text{Score}_{\{t,j\}} = W_{\{t,j\}}^2 / [H^{-1}]_{\{jj\}} \text{ where } H = 2XX^T + \lambda I$$

KL Divergence

$$D_{KL}(P||Q) = \sum p(x) \ln(p(x)/q(x))$$

KD Total Loss

$$L = \alpha \cdot L_{\text{student}} + (1-\alpha) \cdot L_{\text{distill}}$$

Softmax with Temperature

$$q_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$$

Adam optimizer (combined)

$$w \ += \ -\alpha \cdot V_{\square}_t / (\sqrt{\square}_t + \epsilon) \text{ where } V_{\square} = \text{1st moment}, \square = \text{2nd moment}$$

Ring AllReduce cost/GPU

$$2N(P-1)/P \approx 2N \text{ for large } P$$

Scaling Efficiency (Strong)

$$E = T_{\square} / (P \times T_P)$$

SM Occupancy

$$\text{Active Warps per SM} / \text{Max Warps per SM}$$

Roofline FLOPS bound

$$\text{FLOPS} \leq \min(\text{Peak FLOPS}, \text{AI} \times \text{Memory BW})$$

Arithmetic Intensity

$$\text{AI} = \text{\#FLOP} / \text{DRAM_bytes} [\text{FLOP/byte}]$$

LoRA weight update

$$W' = W + AB \text{ where } A \in \mathbb{R}^{(d \times r)}, B \in \mathbb{R}^{(r \times k)}, r \ll d, k$$

Constants & Threshold Values

Constant	Value
Warp size	32 threads (always)
Max threads per block	1024
Shared memory banks	32 banks, 4-byte granularity
FP32 → FP16 reduction	2× memory
FP32 → INT8 reduction	4× memory
FP32 → INT4 reduction	8× memory
Adam default $\beta_{\square}$	0.9
Adam default $\beta_{\square}$	0.95–0.999 (LLMs: 0.95)

AdamW weight decay	0.1 (LLaMA)
FP32 exponent bits	8
FP16 exponent bits	5
BF16 exponent bits	8 (same as FP32 — same range)
FP8 E4M3 max value	448
FP8 E5M2 max value	57,344
INT8 range (signed)	[−128, 127]
BERT training speedup (FlashAttention)	~15% faster (20→17.4 min MLPerf)
2:4 sparsity hardware speedup	Up to 2× on A100 Sparse Tensor Cores
Minitron training cost reduction	40× fewer tokens than from scratch
LLaMA 2 optimizer	AdamW, $\beta_1=0.9$, $\beta_2=0.95$, LR=3e-4 cosine
DeepSeek-V3 total/active params	671B total, 37B active per token
ZeRO-3 memory reduction (N=64)	~64× — from 120 GB to ~2 GB per GPU
vLLM throughput gain	2–4× over static batching
Speculative decoding speedup	2–4× end-to-end